Walk-throughs with Playwright

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.10.0, 2026-10-21

Table of Contents

Colophon . . . 1

Preface . . . 3

Who this book is for . . . 3

A book that stands on its own . . . 3

What you need . . . 4

Conventions . . . 4

I: The Why . . . 5

1. A picture book and a trace . . . 6

1.1. Screenshots . . . 6

1.2. A trace . . . 6

1.3. Readers . . . 7

2. Decisions . . . 8

2.1. Playwright . . . 8

2.2. Steps are spans . . . 8

2.3. The trace goes to the server only . . . 8

2.4. The browser on the machine . . . 9

2.5. Playwright on the module path . . . 9

2.6. One thread . . . 9

II: The How . . . 10

3. Getting started . . . 11

3.1. Adding the module . . . 11

3.2. A browser . . . 11

3.3. Tests on the module path . . . 12

4. Sessions and steps . . . 13

4.1. The application . . . 13

4.2. A walk-through . . . 15

4.3. Options . . . 17

4.4. The session . . . 17

5. What a walk-through records . . . 19

5.1. The model . . . 19

5.2. Screenshots in the record . . . 20

5.3. The browser's console . . . 20

5.4. Failures . . . 21
6. Propagation and origins . . . **23**
6.1. How the browser joins the trace . . . 23
6.2. Other origins . . . 23
6.3. Without an SDK . . . 24
7. A React application . . . **25**
III: Looking Ahead . . . **27**
8. From walk-throughs to documents . . . **28**
Appendix A: Reference . . . **29**
A.1. Types . . . 29
A.2. Spans . . . 29
A.3. Log records . . . 29
Appendix B: Building this book . . . **31**
Appendix C: Glossary . . . **32**

Colophon

Walk-throughs with Playwright: A Nasdanika Zephyr Book

Edition 2026.10.0, built on 2026-10-21.

This edition describes the Playwright module of the Nasdanika SDK Runtime 2026.10.0, Maven coordinates `org.nasdanika.sdk.runtime:playwright:2026.10.0`, Java module `org.nasdanika.sdk.runtime.playwright`, built on Playwright for Java 1.63. Every listing in it is included from the module's sources and tests, or from the runtime's React model tests, which are compiled and run when the book is built. Its figures are screenshots taken by those tests.

Copyright © 2026 Pavel Vlasov.

The text and figures of this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt them for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Walk-throughs with Playwright: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Playwright, TypeScript and Microsoft Edge are trademarks of Microsoft Corporation. Chrome and Chromium are trademarks of Google LLC. Selenium is a trademark of the Software Freedom Conservancy. React is a trademark of Meta Platforms. Java is a trademark of Oracle and its affiliates. OpenTelemetry is a trademark of the Linux Foundation. Other names may be trademarks of their owners. Their use here identifies the technologies discussed and implies no endorsement.

The source of this book, its tests and the module it describes are at github.com/Nasdanika-SDK/

runtime, in `playwright`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about walking through a web application in a browser, from Java, and keeping everything the walk-through shows: what each step looked like, and what the server did for it. It describes the Playwright module of the Nasdanika SDK Runtime. The module drives a browser with Playwright against a server started by the runtime's harness, and records the session in the server's OpenTelemetry trace: each step is a span, the server's spans for the step's requests are its children, and the screenshots taken before and after it are log records in it, with the paths of the image files.

The first part is the why: why a browser test should leave a picture book and a trace, and the decisions behind the module. The second part is the how, with a small catalog application and a React application generated from Java models. The third says what comes next.

Who this book is for

Java developers who test web applications end to end, and want a failed browser test to show the page and the server's side of the failure together.

People who document web applications: a recorded walk-through is a user guide whose screenshots are retaken on every build.

Architects who want traceability from a user's step in a browser to the code that served it. Part one is for you, and needs no code.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. The server side has a Zephyr Book of its own, *Harness*, and the runtime has a book, *Nasdanika SDK Runtime: More Structure with Less Effort*. You do not need either to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do

with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- **A browser**: Google Chrome or Microsoft Edge installed on the machine, or the Chromium that Playwright downloads on first use.
- The module is `org.nasdanika.sdk.runtime.playwright`, Maven coordinates `org.nasdanika.sdk.runtime:playwright:2026.10.0`.
- To record walk-throughs as models, the runtime's telemetry model exporters, `org.nasdanika.sdk.runtime.models.telemetry:exporters`, and for YAML files the runtime's `emf-json` module.

Conventions

- **Every listing is included from the module's sources or tests**, or from a test resource the tests serve, and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **The figures are screenshots the tests took.**
- **Status labels** mark what this edition does not have yet: **PLANNED** for designed but not built.
- **Voice.** I write in the first person.



Editor's notes record inconsistencies and claims to verify found while writing. They are shown in builds for the author and left out of published editions.

Part I: The Why

Why a browser test should leave a picture book and a trace, and the decisions behind the module. No code is needed to read this part.

Chapter 1. A picture book and a trace

A browser test does what a user does: it opens a page, types, clicks, and checks what appears. When it fails, the question is always the same: what did the page look like, and what happened behind it? The first half of the answer is a screenshot; the second half is on the server.

1.1. Screenshots

Screenshots of every step turn a test into a picture book of the application. In an earlier generation of Nasdanika, WebTest did that: it reported each action of a browser test with a screenshot before it and one after, and its reports were read by people who never read the tests. Testers saw where a test went wrong; analysts saw that a feature looked as specified; writers took the pictures for user guides.

What a screenshot cannot show is the server's side: the requests a step sent, what the server did for each, how long it took, what it logged.



Describe Nasdanika WebTest from its repository, github.com/Nasdanika/webtest, before publishing: what it was built on and what its reports contained. The text follows the author's recollection.

1.2. A trace

The runtime's harness starts a server for the length of a test, with every request a span in the test's OpenTelemetry trace. This module puts the browser in the same trace. The session runs in a span; each step is a child span; every request the browser sends to the server carries the step's span, so the server's spans are children of the step that caused them; and each screenshot is a log record in the step, with the image file's path as an attribute.

Recorded with the runtime's telemetry model, the trace is a model of the walk-through:

- **steps**, in order, with what each looked like before and after;
- **requests**, under the step that made them, with what the server did, logged and threw;

- **the browser's console** and the page's uncaught errors, in the step they happened in.

1.3. Readers

Developers debugging a failure read the failing step: its screenshot of the failure, the requests it made, and the server's exception in the same tree.

Reviewers compare the walk-through of a change with the walk-through before it: a step that now makes three requests instead of one, a screenshot that changed.

Writers generate user guides, slide decks and sequence diagrams from the model, a section or a slide per step, with screenshots retaken on every build, so the guide shows the application as it is, not as it was when the guide was written.

Assistants read the same model: a typed record of a scenario, with pictures, that an AI assistant can be given instead of a description, and that tells it what the server did, which no screenshot can.

Chapter 2. Decisions

2.1. Playwright

Playwright drives Chromium, Firefox and WebKit through one API, waits for the page by itself before acting, and has an official Java library. Selenium WebDriver, the older standard, does the same over a W3C protocol, with drivers maintained by the browser vendors. Either would serve; Playwright's automatic waiting makes steps short, and its request routing is what puts the trace context on the browser's requests (Chapter 6).

The module does not wrap Playwright. A session hands out Playwright's own `Page`, `BrowserContext`, `Browser` and `Playwright`, and everything Playwright can do, the test does with them. What the module adds is around the calls: the span of the session, the spans of the steps, the screenshots, the trace context on requests, and the browser's console in the record.

2.2. Steps are spans

A test could be described by a model of its own: a test run of steps of interactions, a navigation, a click, a fill, with screenshots and network requests. I designed one, and decided against building it first.

A step is an operation with a name, a start, an end, a status and children, which is what a span is. A screenshot is something that happened in a step, with attributes, which is what a log record is. A request to the server is a span already, on the server's side. The telemetry model holds all of it, and the harness and the runtime's launcher record into it already. A model of walk-throughs, or of user guides, can be derived from the trace by a transformation when it is needed, and the trace stays the source.

2.3. The trace goes to the server only

The browser's requests to the harness server carry the trace context in their headers. Requests to other origins, a content delivery network serving React or Bootstrap, an analytics service, do not: they have no use for it, it would tell them the trace identifiers of a test, and a header they

did not expect could change how they answer. Playwright's request routing makes the distinction exact: the module routes the server's origin and nothing else (Chapter 6).

2.4. The browser on the machine

Playwright downloads its own builds of the browsers on first use, a few hundred megabytes, which offline builds and locked-down machines cannot do. It can also drive a browser installed on the machine, a *channel*: Google Chrome or Microsoft Edge. The module's options take a channel, and when one is given, the download is skipped. The module's tests use Chrome by default, and are skipped, not failed, when no browser can be launched.

2.5. Playwright on the module path

Playwright's jars are not modules; their manifests name them, `com.microsoft.playwright` and its driver, so they are automatic modules on the module path. The module requires `com.microsoft.playwright` transitively, so a module that requires it can use Playwright's API directly. The driver, a Node.js program with Playwright's browser automation, is in a jar of about two hundred megabytes for all platforms, which Playwright extracts on first use.

2.6. One thread

Playwright for Java is not thread-safe, and it calls event handlers, a route handler or a console listener, on the thread that is waiting in a Playwright call. For the module that is a feature: when the browser sends a request during `page.navigate`, the route handler runs on the test's thread, where the current span is the step's, and the request carries the step. The same is true of console messages and screenshots, so each lands in the step it happened in.

Part II: The How

Adding the module and a browser, walking through an application in steps, reading what the walk-through recorded, and walking through a React application generated from Java models.

Chapter 3. Getting started

3.1. Adding the module

The runtime has a bill of materials, `org.nasdanika.sdk.runtime:bom`, which manages the versions of its modules and their dependencies, Playwright's included. Import it in `dependencyManagement` with the scope `import` and the type `pom`, and depend on `org.nasdanika.sdk.runtime:playwright` without a version, usually in the `test` scope. The module brings the runtime's harness and Playwright for Java, with its driver for every platform, about two hundred megabytes.

To record walk-throughs as models, add the runtime's telemetry model exporters, `org.nasdanika.sdk.runtime.models.telemetry:exporters`, and for YAML files the runtime's `emf-json` module.

Listing 1. The module declaration (`module-info.java`)

```
/**
 * Browser sessions with Playwright against a harness server, in the server's trace, with
 * screenshots recorded in the spans they were taken in.
 */
@SuppressWarnings("requires-transitive-automatic") // Playwright is an automatic module
module org.nasdanika.sdk.runtime.playwright {

    requires transitive org.nasdanika.sdk.runtime.harness;
    requires transitive com.microsoft.playwright;

    exports org.nasdanika.sdk.runtime.playwright;

}
```

3.2. A browser

A session launches Chromium, in one of two ways:

- With a **channel**, a browser installed on the machine: `chrome`, `msedge`, or their beta and dev channels. Nothing is downloaded.
- Without one, the **Chromium Playwright downloads** on first use into a cache in the user's home directory, a few hundred megabytes, once.

The module's tests take the channel from the `nasdanika.playwright.channel` system property, which the build sets from the `playwright.channel` Maven property, `chrome` by default; an empty value means Playwright's Chromium. A test should be skipped, not failed, on a machine without a browser, and the module's tests check that one can be launched before they start:

Listing 2. Skipping without a browser (`WalkthroughTests.java`)

```
@BeforeAll
static void browserAvailable() {
    Options options = options(Path.of("target"));
    Map<String, String> environment = options.channel() == null
        ? Map.of()
        : Map.of("PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD", "1"); // An installed browser
    var create = new Playwright.CreateOptions().setEnv(environment);
    var launch = new BrowserType.LaunchOptions().setChannel(options.channel());
    try (Playwright playwright = Playwright.create(create);
            Browser browser = playwright.chromium().launch(launch)) {
        // Available
    } catch (RuntimeException e) {
        Assumptions.abort("No browser to drive: " + e.getMessage());
    }
}
```

3.3. Tests on the module path

Tests patched into a module on the module path read what the module reads. Maven's Surefire plugin puts the telemetry model on the module path and its exporters, which need the OpenTelemetry SDK, on the class path, so a test that reads a recording needs one more read edge, `--add-reads <your module>=org.nasdanika.sdk.runtime.models.telemetry`, in the Surefire configuration.

Chapter 4. Sessions and steps

4.1. The application

The examples walk through a catalog of books: a page with a search box, a list of titles, and the details of the title clicked.

Listing 3. The page (`catalog/index.html`)

```
<h1>Catalog</h1>
<label>Title <input id="title" type="search"></label>
<ul id="books"></ul>
<section id="details" hidden>
  <h2 id="book-title"></h2>
  <p id="book-author"></p>
</section>
<script src="catalog.js"></script>
```

Listing 4. Its script (`catalog/catalog.js`)

```
const list = document.getElementById('books');
const details = document.getElementById('details');

async function search(text) {
  const response = await fetch('/data/books?q=' + encodeURIComponent(text));
  const books = await response.json();
  list.replaceChildren(...books.map(book => {
    const link = document.createElement('a');
    link.href = '#' + book.id;
    link.textContent = book.title;
    link.addEventListener('click', () => show(book.id));
    const item = document.createElement('li');
    item.append(link);
    return item;
  }));
}

async function show(id) {
  const response = await fetch('/data/books/' + id);
  const book = await response.json();
  document.getElementById('book-title').textContent = book.title;
  document.getElementById('book-author').textContent = book.author;
  details.hidden = false;
}

document.getElementById('title').addEventListener('input', event => search(event.target.value));
```

```
search('').then(() => console.log('Catalog ready'));
```

The server is a harness server with two routes:

Listing 5. The routes (`Catalog.java`)

```
static void routes(HttpServer routes) {
    Path site = Path.of("src", "test", "resources", "catalog");
    routes.createContext("/", new VolumeHttpHandler(Volume.of(site))); ①
    routes.createContext("/data", Catalog::data); ②
}
```

① The page, its script and its stylesheet, served from a directory by the runtime's volume handler.

② The data, computed per request.

Listing 6. The data

```
static void data(HttpExchange exchange) throws IOException {
    var logger = Telemetry.logger(Telemetry.current());
    String path = exchange.getRequestURI().getPath();
    if (path.equals("/data/books")) {
        String query = query(exchange);
        Telemetry.log(logger, Severity.INFO, null, "Searching for '" + query + "'", null);
        String books = BOOKS.stream()
            .filter(book -> book.title().toLowerCase(Locale.ROOT).contains(query))
            .map(Book::json)
            .collect(Collectors.joining(",", "[", "]"));
        json(exchange, 200, books);
    } else {
        String id = path.substring("/data/books/".length());
        Telemetry.log(logger, Severity.INFO, null, "Showing " + id, null);
        BOOKS.stream()
            .filter(book -> book.id().equals(id))
            .findFirst()
            .ifPresentOrElse(
                book -> json(exchange, 200, book.json()),
                () -> json(exchange, 404, "{}"));
    }
}
```

The log records are written in the handler, so they go into the server span of the request.

4.2. A walk-through

`Browsing.inBrowser` runs a browser session against a harness server:

Listing 7. Three steps through the catalog (`WalkthroughTests.java`)

```
Recording recording = Recording.builder()
    .serviceName("catalog")
    .directory(dir, "yml") ①
    .build();
try (recording) {
    Harness.inServer(recording.getOpenTelemetry(), "Catalog", Catalog::routes, server ->
        Browsing.inBrowser(server, options(dir), session -> { ②
            Page page = session.page();
            session.step("Open the catalog", () -> {
                page.navigate("/index.html"); ③
                assertThat(page.locator("#books li")).hasCount(4);
            });
            session.step("Search for Dune", () -> {
                page.getByLabel("Title").fill("dune");
                assertThat(page.locator("#books li")).hasCount(1); ④
            });
            session.step("Open Dune", () -> {
                page.getByText("Dune").click();
                assertThat(page.locator("#book-author")).hasText("Frank Herbert");
            });
            return null;
        }));
}
```

① The recording: the trace, and the screenshots next to it, in `target/walkthrough`.

② A session against the server: a browser, a context whose relative URLs resolve against the server's base, and a page, in a `Browser` span.

③ A relative URL: the server's page.

④ Playwright's assertions wait for the page: the list has one item once the server has answered the search.

Each step is a span named after it. With the default options, a step takes a screenshot before its body runs and another after, except before the first page is loaded, when there is nothing to see.

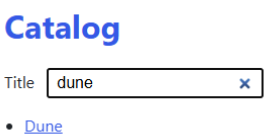


Figure 1. After the second step (`003-search-for-dune-after.png`)

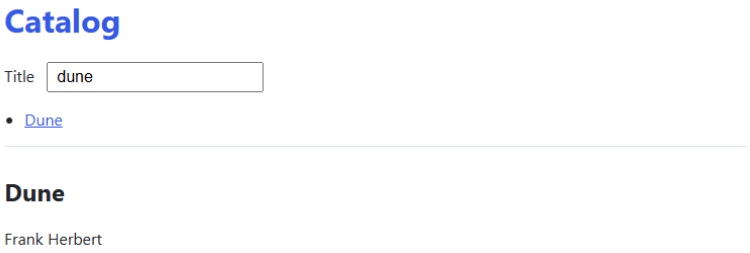


Figure 2. After the third step (`005-open-dune-after.png`)

A step's body ends when its last Playwright call returns. Waiting for what the step does, with an assertion or `waitFor`, inside the step, keeps the step's requests in the step: a request the page sends after the step ended belongs to whatever is current then.

4.3. Options

`Options` is a record with a `with` method per component, and `Options.DEFAULT`:

Component	Default	Meaning
`channel`	none	An installed browser, such as `chrome` or `msedge`; none for Playwright's Chromium
`headless`	`true`	Without a window
`width`, `height`	1280, 800	The viewport, in CSS pixels
`screenshots`	`target/screenshots`	The directory screenshots are written to, created if missing
`stepScreenshots`	`BEFORE_AND_AFTER`	What a step takes by itself: `NONE`, `AFTER`, or `BEFORE_AND_AFTER`
`fullPage`	`false`	The full scrollable page rather than the viewport

Screenshots are named by their position in the session, the name of the step or screenshot, and the phase: `003-search-for-dune-after.png`. Writing them to the recording's directory keeps the trace and its pictures together, and their paths, relative to the directory, valid wherever the directory is copied.

4.4. The session

The body receives a `Session`:

Method	Returns or does
`page()`	The page, Playwright's `Page`
`context()`, `browser()`, `playwright()`	The rest of Playwright
`server()`	The harness server
`options()`	The options
`step(name, body)`	Runs the body in a span, with screenshots

Method	Returns or does
`screenshot(name)`	Takes a screenshot now, in the current span, and returns the file

Chapter 5. What a walk-through records

5.1. The model

After the recording is closed, the walk-through is a model, and a test can read it:

Listing 8. A step, its screenshots and the server's work (`WalkthroughTests.java`)

```
Span search = span(recording, "Search for Dune");
assertThat(search.getLogRecords()) ①
    .extracting(log -> attribute(log, Browsing.SCREENSHOT_PATH.getKey()))
    .containsExactly("002-search-for-dune-before.png", "003-search-for-dune-after.png");

Span served = search.getChildren().getFirst(); ②
assertThat(served.getName()).isEqualTo("GET /data");
assertThat(served.getKind()).isEqualTo(SpanKind.SPAN_KIND_SERVER);
assertThat(served.getLogRecords())
    .singleElement()
    .extracting(log -> log.getBody().getStringValue())
    .isEqualTo("Searching for 'dune'");
```

① The step's log records include its screenshots, with the files' paths.

② The step's child is the server's span for the search, with the handler's log record.

The test also reduces the trace to an outline of what does not change from run to run, and compares it with this file:

Listing 9. The walk-through in outline (`walkthrough-outline.txt`)

```
Catalog
  Browser
    Open the catalog
      screenshot 001-open-the-catalog-after.png
      console Catalog ready
      GET /data: Searching for ''
    Search for Dune
      screenshot 002-search-for-dune-before.png
      screenshot 003-search-for-dune-after.png
      GET /data: Searching for 'dune'
    Open Dune
      screenshot 004-open-dune-before.png
      screenshot 005-open-dune-after.png
      GET /data: Showing dune
```

The outline leaves out the requests for the page, its script and its stylesheet, which the browser sends in parallel, and identifiers, timestamps and attributes. The full record is in `target/walkthrough/traces.yml`: every span with its timing and attributes, every request the server handled, under the step that sent it.

5.2. Screenshots in the record

A screenshot is a log record with the event name `nasdanika.screenshot`, in the span that was current when it was taken, with these attributes:

Attribute	Value
`nasdanika.screenshot.name`	The step's name, or the name given to `screenshot`
`nasdanika.screenshot.path`	The file, relative to the screenshots directory
`nasdanika.screenshot.phase`	`before`, `after` or `failure` for a step's own, `explicit` for one taken with `screenshot`
`nasdanika.screenshot.index`	The position in the session, from 1
`url.full`	The page's URL

The body of the record is the name, so a log viewer that knows nothing of screenshots still shows something readable.

5.3. The browser's console

Console messages are log records with the event name `nasdanika.browser.console`, the message as the body and the page's URL as an attribute, with a severity by the message's type: `error` is an error, `warning` a warning, `debug` debug, anything else information. Uncaught errors in the page are log records with the event name `nasdanika.browser.page_error` and the error severity. They are in the span that was current when Playwright delivered them, usually the step that caused them.

5.4. Failures

A step whose body throws takes a screenshot of the failure, records the exception on its span, and rethrows:

Listing 10. A step that fails

```
Recording recording = Recording.builder().directory(dir, "yml").build();
try (recording) {
    Harness.inServer(recording.getOpenTelemetry(), "Catalog", Catalog::routes, server ->
        Browsing.inBrowser(server, options(dir), session -> {
            session.step("Open the catalog", () -> session.page().navigate("/index.html"));
            assertThatThrownBy(() -> session.step("Click a missing button", () -> {
                Locator missing = session.page().locator("#missing");
                missing.click(new Locator.ClickOptions().setTimeout(500));
            })).hasMessageContaining("Timeout");
            return null;
        }));
}
Span failed = span(recording, "Click a missing button");
assertThat(failed.getStatus().getCode()).isEqualTo(StatusCode.STATUS_CODE_ERROR); ①
assertThat(failed.getLogRecords())
    .extracting(log -> attribute(log, Browsing.SCREENSHOT_PHASE.getKey()))
    .containsExactly("before", "failure"); ②
```

① The step's span has the error status and the exception.

② Its screenshots: before the step, and at the failure.

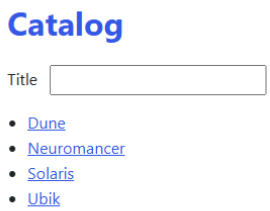


Figure 3. The failure (`003-click-a-missing-button-failure.png`)

A failure of the body of the session is recorded on the `Browser` span, and of the harness body on its span, so the error shows at every level of the tree, with the step where it started at the bottom.

Chapter 6. Propagation and origins

6.1. How the browser joins the trace

The browser does not know about OpenTelemetry; Playwright tells it what to send. When the session starts, the module routes every request to the server's origin, `http://127.0.0.1:<port>/**`, through a handler that adds the trace context of the current span to the request's headers and lets the request go on. Playwright calls the handler on the thread that is waiting in a Playwright call, the test's thread, where the current span is the step that is running, so the server's span for the request is a child of the step.

A route applies to every request from the page: the document, scripts, stylesheets, images, `fetch` calls. Routed requests bypass the browser's HTTP cache, which for a test is what you want: every step's requests reach the server and are in the trace.

6.2. Other origins

Requests to any other origin are not routed and get no trace context:

Listing 11. A request to another origin (`WalkthroughTests.java`)

```
List<String> received = new CopyOnWriteArrayList<>();
Recording recording = Recording.builder().directory(dir, "yml").build();
try (recording) {
    OpenTelemetry otel = recording.getOpenTelemetry();
    Harness.inServer(otel, "Other origin", other -> other.createContext("/", exchange -> {
        received.add(String.valueOf(exchange.getRequestHeaders().getFirst("traceparent")));
        exchange.getResponseHeaders().set("Access-Control-Allow-Origin", "*");
        exchange.sendResponseHeaders(204, -1);
        exchange.close();
    }), otherServer -> Harness.inServer(otel, "Catalog", Catalog::routes, server ->
        Browsing.inBrowser(server, options(dir), session -> {
            session.page().navigate("/index.html");
            String ping = otherServer.uri("/ping").toString();
            session.page().evaluate("url => fetch(url).then(r => r.status)", ping); ①
            return null;
        })));
}
assertThat(received).containsExactly("null"); ②
assertThat(span(recording, "Browser").getChildren()) ③
    .extracting(Span::getName)
```

```
    .contains("GET /");
```

① The page fetches a URL of another server, another port, which is another origin.

② That server received no `traceparent`.

③ The page itself was served in the session's trace.

The second server here is a harness server too, which shows something else: harness bodies nest, and two servers can run in one test, one trace or several.

6.3. Without an SDK

With the no-op OpenTelemetry instance there is no span to propagate, the headers are empty, and the routing passes requests through unchanged. A walk-through without a recording still takes its screenshots.

Chapter 7. A React application

The runtime generates React applications from Java models: a component is an object of the React model, its markup objects of the HTML model, and the generator writes an ES module that a browser runs as it is, with React loaded from a content delivery network through an import map. No npm, no Node.js, no build step. The runtime's React model tests generate a table of the properties of the running JVM, and serve the page, the module and the data, computed per request, from a harness server.

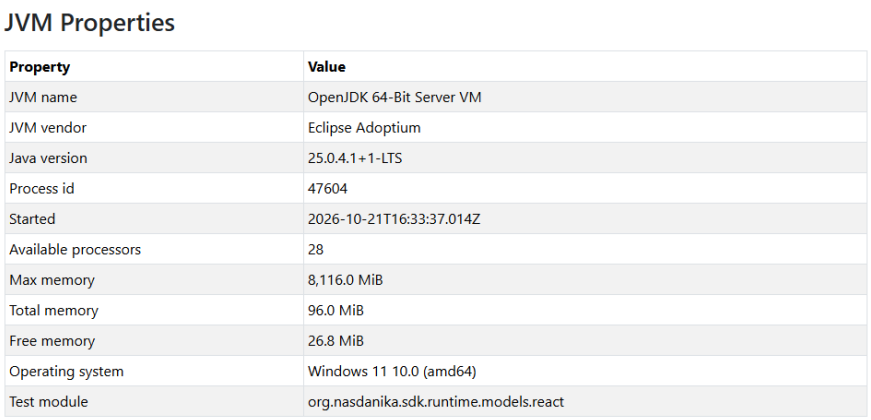
This walk-through is in those tests:

Listing 12. Walking through a generated React application (`JvmPropertiesWalkthroughTests.java`, React model)

```
Recording recording = Recording.builder()
        .serviceName("jvm-properties")
        .directory(dir, "yml")
        .build();
try (recording) {
    OpenTelemetry otel = recording.getOpenTelemetry();
    Harness.inServer(otel, "JVM properties", JvmPropertiesTableTests::routes, server -> ①
        Browsing.inBrowser(server, options(dir), session -> {
            Page page = session.page();
            session.step("Open the JVM properties", () -> {
                page.navigate("/index.html");
                page.getByText("Process id").waitFor(); ②
            });
            session.step("Reload for fresh numbers", () -> {
                page.reload();
                page.getByText("Free memory").waitFor();
            });
            return null;
        }));
}
```

① The routes of the React tests: the page, the generated module and the JSON data.

② The table is rendered once React has loaded and the data has arrived.



JVM Properties

Property	Value
JVM name	OpenJDK 64-Bit Server VM
JVM vendor	Eclipse Adoptium
Java version	25.0.4.1+1-LTS
Process id	47604
Started	2026-10-21T16:33:37.014Z
Available processors	28
Max memory	8,116.0 MiB
Total memory	96.0 MiB
Free memory	26.8 MiB
Operating system	Windows 11 10.0 (amd64)
Test module	org.nasdanika.sdk.runtime.models.react

Figure 4. The generated application, after the first step

In the recorded trace, the first step has the server's spans for the page, `GET /`, the module, `GET /modules/`, and the data, `GET /data/`, as its children. React, React Bootstrap and Bootstrap's stylesheet came from content delivery networks, without the trace context. The second step reloads the page, and its data span shows the JVM's memory a moment later.

The test needs internet access for the content delivery networks, and checks for it before it starts, as it checks for a browser.

So a JVM generates the application, serves it, drives it in a browser and records it, and the record joins the browser's steps to the server's work. JavaScript is what the browser runs; everything around it is Java and models.

Part III: Looking Ahead

From recorded walk-throughs to documents, decks and diagrams.

Chapter 8. From walk-throughs to documents

A recorded walk-through holds what a document about a scenario needs: the steps in order, named; what each looked like; what each asked of the server; what went wrong, where. PLANNED:

- **User guides in AsciiDoc**: a section per step with its "after" screenshot, generated from the trace, and rebuilt with the application, so the pictures are never out of date.
- **Slide decks**: a slide per step, through the runtime's presentation and PowerPoint models, for demos and reviews.
- **Sequence diagrams** of a step: the browser, the server and what the server called, from the tree of spans, laid out with the runtime's diagram model and ELK.
- **Process flows** of a scenario: steps as activities, generated as a diagram.
- **Comparisons of runs**: steps added or removed, requests added, slower or failing, screenshots that changed.
- **Narrated video**: the screenshots of a walk-through as frames of the runtime's planned video model, with narration from the steps' names and documentation.

The trace is the source of all of them, so each is a transformation of one model into another, and none needs the test to know about it.

Appendix A: Reference

A.1. Types

Type	Purpose
`Browsing`	`inBrowser(server, body)` with the default options, `inBrowser(server, options, body)`; the attribute keys and event names below
`Browsing.BrowserBody<T, E>`	The body: `T run(Session session) throws E`
`Session`	The Playwright objects, `step(name, body)` and `screenshot(name)` (Chapter 4)
`Session.StepBody<E>`	A step's body: `void run() throws E`
`Options`	The browser channel, headless mode, viewport, screenshots directory, step screenshots and full-page screenshots
`Options.StepScreenshots`	`NONE`, `AFTER`, `BEFORE_AND_AFTER`

A.2. Spans

Span	Kind	Attributes
`Browser`, the session	`INTERNAL`	`browser.name`, `browser.version`, `nasdanika.browser.channel` when a channel is used
A step, named by the caller	`INTERNAL`	`nasdanika.step.index`

The server's spans for the browser's requests are children of the span current when the browser sent them: a step, or the `Browser` span between steps.

A.3. Log records

Event name	Severity	Body and attributes
`nasdanika.screenshot`	Info	The name; `nasdanika.screenshot.name`, `.path`, `.phase`, `.index`, `url.full`

Event name	Severity	Body and attributes
`nasdanika.browser.console`	By the message's type	The message; `url.full`
`nasdanika.browser.page_error`	Error	The error; `url.full`

Appendix B: Building this book

This book is AsciiDoc in `playwright/src/docs/asciidoc`, built with the module after its tests have passed:

```
mvn -pl playwright verify                                # the module, its tests and this book
mvn -pl playwright verify -Dplaywright.channel=msedge    # tests with Microsoft Edge
mvn -pl playwright verify -Dplaywright.channel=          # tests with Playwright's Chromium
mvn -pl playwright verify -Dzephyr.editor-notes=false    # without editor's notes
mvn -pl playwright verify -Dasciidoctor.skip=true        # without the book
```

The PDF, the single-page HTML and the EPUB are written to `target/book`; the PDF and the HTML are attached to the module's artifact with the classifier `book`, and the test sources with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/playwright/tests`, of the catalog application in `src/test/resources/catalog`, of the module's `module-info.java`, and of the React model's walk-through test. The tests record their walk-throughs, with screenshots, in `target/walkthrough`, `target/failing-step` and `target/origins`. The figures are copies of screenshots from those runs, in `src/test/resources/figures`, so that the book builds when the tests are skipped. A listing line holds at most 96 characters, and a warning fails the build.

Appendix C: Glossary

Channel

A browser installed on the machine that Playwright can drive, such as `chrome` or `msedge`.

Harness server

An HTTP server that is up while code runs against it, with every request a span in the code's trace.

Origin

The scheme, host and port of a URL. The trace context is sent to the harness server's origin only.

Propagation

Carrying a trace context to another component in request headers, `traceparent` and `baggage`.

Recording

An OpenTelemetry SDK that exports to the runtime's telemetry model and saves it when closed.

Route

A Playwright handler for the requests a page sends to matching URLs, which can change them before they go on.

Session

A browser, a context and a page, used by a body in a `Browser` span.

Step

A named part of a session, run in a span, with screenshots before, after, or at a failure.

Walk-through

A recorded session: its steps, what each looked like, and the server's work for each.